

Seoul Bike Rental Prediction using Machine Learning

Project Documentation

Submitted by: Apeetha S

Batch - DSB02

Course: Data Science Certification Course

Project Type: Mini Project

Tools & Technologies: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, Flask, Docker

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Dataset Description
5. Project Workflow
6. Implementation
 - 6.1 Importing Required Libraries
 - 6.2 Dataset Loading and Initial Exploration
 - 6.3 Data Preprocessing
 - 6.4 Exploratory Data Analysis
 - 6.5 Feature Engineering
 - 6.6 Feature Selection
 - 6.7 Data Preparation
 - 6.8 Model Building
 - 6.9 Hyperparameter Tuning
 - 6.10 Model Evaluation
 - 6.11 Model Deployment
7. Results
8. Conclusion
9. Future Scope

1. Project Overview

Bike-sharing systems have become an integral part of modern urban transportation by providing an economical, convenient, and environmentally friendly mode of travel. Accurate prediction of bike rental demand enables service providers to optimize bike distribution, reduce operational costs, and improve customer satisfaction.

This project focuses on developing a machine learning model capable of predicting the number of bikes rented in Seoul based on various environmental and temporal factors. The dataset includes historical records containing weather conditions, seasonal information, holidays, and time-related attributes that significantly influence bike rental demand.

The project follows a complete machine learning pipeline beginning with data collection and preprocessing, followed by exploratory data analysis, feature engineering, feature selection, model training, hyperparameter tuning, and performance evaluation. Multiple regression algorithms are compared to identify the best-performing model. Finally, the selected model is deployed as a web application using Flask and Docker, enabling users to generate real-time predictions through an interactive interface.

Detailed implementation code, visualizations, and analytical observations are provided in the accompanying Jupyter Notebook.

2. Problem Statement

The demand for public bike-sharing services fluctuates significantly due to weather conditions, seasonal variations, holidays, and time-dependent factors. Predicting bike rental demand accurately is challenging because multiple variables simultaneously influence user behavior.

The objective of this project is to build a machine learning regression model capable of accurately predicting the number of bikes rented in Seoul using historical rental data and associated environmental attributes. Such predictions can assist bike-sharing operators in improving resource allocation, minimizing shortages or surpluses, and enhancing overall operational efficiency.

3. Objectives

The primary objectives of this project are:

- To analyze the Seoul Bike Rental dataset and understand the factors influencing bike rental demand.
- To perform data preprocessing and handle data quality issues.

- To conduct exploratory data analysis for identifying trends, patterns, and relationships among variables.
- To engineer and select the most relevant features for model training.
- To train and compare multiple regression algorithms.
- To improve model performance using hyperparameter tuning.
- To evaluate each model using appropriate regression performance metrics.
- To select the best-performing model for deployment.
- To develop a Flask-based web application capable of generating real-time bike rental predictions.
- To containerize the application using Docker for consistent deployment across different environments.

4. Dataset Description

The dataset used in this project is the **Seoul Bike Sharing Demand Dataset**, which contains historical bike rental information collected from the Seoul public bike-sharing system.

The dataset includes weather conditions, seasonal information, holidays, and hourly rental records. These variables serve as predictors for estimating the number of bikes rented during a particular hour.

Dataset Characteristics

- **Dataset Name:** Seoul Bike Sharing Demand Dataset
- **Problem Type:** Supervised Machine Learning (Regression)
- **Target Variable:** Rented Bike Count
- **File Format:** CSV
- **Number of Records:** 8,760 (hourly observations covering one year)
- **Number of Features:** 14 input attributes and 1 target variable

Key Features

The dataset contains several important attributes, including:

- Date
- Hour
- Temperature (°C)
- Humidity (%)

- Wind Speed (m/s)
- Visibility (10 m)
- Dew Point Temperature (°C)
- Solar Radiation (MJ/m²)
- Rainfall (mm)
- Snowfall (cm)
- Seasons
- Holiday
- Functioning Day
- Rented Bike Count (Target Variable)

These features collectively represent weather conditions, seasonal variations, and operational information that influence bike rental demand.

5. Project Workflow

Data Collection → Data Preprocessing → EDA → Feature Engineering → Feature Selection → Data Preparation → Model Building → Hyperparameter Tuning → Model Evaluation → Deployment

The project follows a structured machine learning workflow consisting of the following stages and the methods and libraries used for implementing it.

Step 1: Data Collection

The Seoul Bike Rental dataset is imported into the Python environment and examined to understand its structure and quality.

Step 2: Data Preprocessing

The dataset is cleaned by checking for missing values, duplicate records, and incorrect data types. Necessary transformations are applied to prepare the data for analysis.

Step 3: Exploratory Data Analysis (EDA)

Various statistical analyses and visualizations are performed to understand feature distributions, identify trends, detect outliers, and examine relationships between variables.

Step 4: Feature Engineering

New features are created from existing variables, and categorical features are encoded into numerical representations suitable for machine learning algorithms.

Step 5: Feature Selection

Relevant features are selected using statistical techniques to improve model performance while reducing unnecessary complexity.

Step 6: Data Preparation

The dataset is divided into training and testing sets. Numerical features are standardized to ensure that models perform effectively.

Step 7: Model Building

Several regression algorithms are trained and compared, including:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- XGBoost Regressor

Step 8: Hyperparameter Tuning

The Decision Tree, Random Forest, and XGBoost models are optimized using hyperparameter tuning techniques to improve predictive accuracy and generalization.

Step 9: Model Evaluation

The performance of each model is evaluated using regression metrics such as:

- R^2 Score
- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

The best-performing model is selected based on these evaluation metrics.

Step 10: Model Deployment

The final XGBoost model, along with the preprocessing components, is deployed using Flask. The application is containerized with Docker, enabling consistent execution and easy deployment across different environments. Users can provide input values through a web interface and obtain real-time bike rental predictions.

6. Implementation

6.1 Importing Required Libraries

The first step in the implementation phase involved importing the necessary Python libraries required for data manipulation, numerical computation, and data visualization.

The following libraries were used:

- **Pandas:** Used for loading, processing, and analyzing structured data in the form of DataFrames.
- **NumPy:** Used for performing numerical operations and handling mathematical computations.
- **Matplotlib:** Used for creating basic data visualizations and graphical representations.
- **Seaborn:** Used for generating advanced statistical visualizations to identify patterns and relationships within the dataset.

These libraries provide the essential functionalities required throughout the project lifecycle, including data preprocessing, exploratory data analysis, feature engineering, and model development.

6.2 Dataset Loading and Initial Data Exploration

6.2.1 Dataset Loading

The Seoul Bike Rental dataset was loaded into the Python environment using the Pandas `read_csv()` function. The dataset was imported as a DataFrame to enable efficient data manipulation and analysis.

Since the dataset contained special characters in some attributes, the `latin1` encoding format was specified while loading the CSV file. This ensured that all records were successfully read without character encoding errors.

The loaded dataset represents historical bike rental records collected from the Seoul public bike-sharing system. It contains information related to rental demand, weather conditions, seasonal factors, and operational attributes.

6.2.2 Dataset Structure Analysis

After loading the dataset, an initial inspection was performed to understand the overall structure and characteristics of the data.

The following exploratory checks were performed:

Dataset Preview

The first and last few records of the dataset were displayed to verify successful loading and obtain an initial understanding of the available attributes and data format.

Dataset Dimensions

The number of rows and columns in the dataset was identified using the shape attribute. This provided information about the total number of observations and features available for analysis.

Data Information

The `info()` function was used to examine:

- Number of records
- Feature names
- Data types of each attribute
- Presence of missing values
- Memory usage of the dataset

This step helped identify numerical, categorical, and date-related variables and guided further preprocessing decisions.

Statistical Analysis

Descriptive statistics were generated for numerical features using the `describe()` function.

The statistical summary provided insights into:

- Central tendency
- Data distribution
- Minimum and maximum values
- Feature variability

The numerical columns were further analyzed using statistical measures such as:

- Mean
- Median
- Standard deviation
- Minimum value
- Maximum value

These measures helped understand the range and distribution of each numerical variable before performing further analysis.

6.2.3 Data Quality Assessment

Before applying machine learning algorithms, the dataset was evaluated for potential data quality issues.

The following validation checks were performed:

Missing Value Analysis

The dataset was examined for null or missing values using the `isnull().sum()` function.

This step ensured that incomplete records were identified before preprocessing and model training.

Duplicate Record Detection

Duplicate observations were identified using the `duplicated().sum()` function.

Removing duplicate records, if present, prevents repeated information from influencing model learning and helps maintain data integrity.

Data Type Verification

The data types of all attributes were checked using the `dtypes` attribute.

This verification ensured that each feature was stored in an appropriate format for analysis and machine learning operations.

6.2.4 Date Format Conversion

The **Date** column was initially stored as an object/string data type. To enable effective time-based analysis, it was converted into a `DateTime` format using the Pandas `to_datetime()` function.

The conversion allows extraction of additional temporal information such as:

- Year
- Month
- Day
- Weekday
- Seasonal patterns

These time-based features can provide valuable insights into variations in bike rental demand.

Outcome

The initial data exploration phase successfully prepared the dataset for further processing. The dataset structure, feature types, statistical characteristics, and data quality were examined. The Date column was converted into an appropriate format, creating a suitable foundation for exploratory data analysis, feature engineering, and machine learning model development.

6.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the characteristics of the Seoul Bike Rental dataset. The analysis focused on studying individual feature distributions, identifying patterns, detecting outliers, and examining relationships between input variables and the target variable (Rented Bike Count).

6.3.1 Univariate Analysis

Univariate analysis was performed to analyze individual variables independently and understand their distribution, frequency, variability, and presence of outliers.

Numerical Feature Distribution Analysis

Histograms were used to visualize the distribution of numerical features:

- **Hour**
- **Temperature (°C)**
- **Humidity (%)**
- **Wind Speed (m/s)**
- **Visibility (10m)**
- **Dew Point Temperature (°C)**
- **Solar Radiation (MJ/m²)**
- **Rainfall (mm)**
- **Snowfall (cm)**

Subplots were used to display multiple distributions together, allowing efficient comparison between different features.

The analysis helped understand:

- **Feature ranges**
 - **Frequency patterns**
 - **Data distribution characteristics**
 - **Weather condition variations**
-

Categorical Feature Distribution Analysis

Count plots were used to analyze categorical variables:

- **Seasons**

- **Holiday**
- **Functioning Day**

This analysis helped understand the distribution of different categories and their impact on bike rental availability.

Skewness Analysis

Skewness was calculated for numerical features to understand the symmetry of their distributions.

This analysis helped identify highly skewed variables and determine whether data transformation techniques might be required during preprocessing.

Outlier Detection

Box plots were created for numerical features to identify potential outliers and understand data variability.

The analysis was performed on features such as:

- **Temperature**
- **Humidity**
- **Wind Speed**
- **Visibility**
- **Solar Radiation**
- **Rainfall**
- **Snowfall**

Outlier analysis helped determine whether additional preprocessing was required before model training.

6.4 Bivariate Analysis

Bivariate analysis was performed to study the relationship between two variables, specifically between the independent features and the target variable (Rented Bike Count).

Relationship Analysis Using Scatter Plots

Scatter plots were used to analyze how numerical features influence bike rental demand.

The following features were compared against Rented Bike Count:

- **Hour**
- **Temperature (°C)**
- **Humidity (%)**
- **Wind Speed (m/s)**
- **Visibility (10m)**
- **Dew Point Temperature (°C)**
- **Solar Radiation (MJ/m²)**
- **Rainfall (mm)**
- **Snowfall (cm)**

The scatter plots helped identify trends, patterns, and the strength of relationships between different environmental factors and bike rental demand.

Key Observations

- **Bike rental demand varies significantly across different hours, indicating time-based usage patterns.**
- **Temperature shows a relationship with rental demand, as comfortable weather conditions encourage cycling.**
- **Humidity, wind speed, rainfall, and snowfall influence user behavior and may reduce rentals under unfavorable conditions.**
- **Weather-related features provide important information for predicting bike rental demand.**

Outcome

The univariate analysis provided insights into individual feature distributions, category frequencies, skewness, and outliers. The bivariate analysis helped understand relationships between predictor variables and the target variable. These findings guided further preprocessing, feature engineering, and model development steps.

6.4.2 Correlation Analysis Using Heatmap

A correlation heatmap was generated to analyze the relationship between numerical features in the dataset. Correlation analysis helps identify the strength and direction of linear relationships between variables.

The Pearson correlation coefficient was calculated for all numerical features and represented using a heatmap with annotated correlation values.

The correlation values range between:

- **+1:** Strong positive correlation
- **0:** No linear relationship
- **-1:** Strong negative correlation

The heatmap was used to identify:

- Features strongly related to the target variable (**Rented Bike Count**)
- Highly correlated independent variables
- Possible multicollinearity among features

Key Observations

- The correlation analysis helped identify which numerical features have a significant relationship with bike rental demand.
- Weather-related variables showed varying levels of correlation with the target variable.
- Features with strong relationships can contribute more effectively during model training.
- Highly correlated features among themselves were analyzed to avoid redundancy during feature selection.

Outcome

The correlation heatmap provided insights into feature relationships and supported feature selection decisions. It helped in understanding the importance of numerical variables before proceeding with model development.

6.5 Feature Engineering

Feature engineering was performed to create additional meaningful features from existing attributes and improve the model's ability to capture time-based patterns in bike rental demand.

Month Extraction

The **Month** feature was extracted from the **Date** column to capture seasonal variations in rental demand.

Weekend Indicator

A new binary feature **Weekend** was created based on the **Day_of_Week** column:

- **0 → Weekdays**
- **1 → Weekends**

This feature helps identify differences in bike rental behavior between weekdays and weekends.

Feature Analysis

The engineered features were analyzed using bar plots:

- Average bike rentals by day of the week
- Average bike rentals by month
- Average bike rentals during weekdays and weekends

These visualizations helped understand temporal patterns and user behavior.

Outcome

Feature engineering added useful time-based information to the dataset, improving its suitability for machine learning model training.

6.6 Categorical Feature Encoding

Machine learning algorithms require numerical input data, so categorical variables were converted into numerical representations during preprocessing.

Label Encoding

Label Encoding was applied to the categorical features:

- **Functioning Day**
- **Holiday**

The categories were transformed into numerical values using LabelEncoder.

The encoding mappings were generated to maintain consistency and understand how each category was represented.

One-Hot Encoding

One-hot encoding was applied to the following categorical features:

- **Seasons**
- **Day_of_Week**

The `get_dummies()` method was used with `drop_first=True` to avoid creating redundant dummy variables and reduce multicollinearity.

Outcome

Categorical features were successfully converted into numerical format, making the dataset suitable for machine learning algorithms.

6.7 Outlier Detection

Outlier detection was performed to identify extreme values that deviate significantly from the normal range of the dataset. Identifying outliers helps understand data quality and their possible impact on machine learning model performance.

The **Interquartile Range (IQR)** method was used to detect outliers in numerical features.

The following features were analyzed:

- Wind Speed (m/s)
- Solar Radiation (MJ/m²)
- Rainfall (mm)
- Snowfall (cm)
- Rented Bike Count

The IQR method calculates the difference between the first quartile (Q1) and third quartile (Q3) to determine the normal data range. Values falling below the lower bound or above the upper bound were identified as potential outliers.

Outcome

The number of outliers present in each feature was calculated and analyzed. This step helped understand extreme values in weather conditions and rental demand before proceeding with model training.

6.8 Feature Selection

Feature selection was performed to identify the most relevant input variables for predicting **Rented Bike Count**. Selecting important features helps improve model performance, reduce computational complexity, and minimize the impact of irrelevant variables.

The **SelectKBest** method with the **f_regression** scoring function was used to evaluate the relationship between each feature and the target variable.

The top **12 most relevant features** were selected based on their statistical scores, while the target variable and the **Dew Point Temperature** feature were excluded from the feature selection process.

Outcome

The selected features were used for subsequent model training, ensuring that the models were built using the most informative predictors for bike rental demand.

6.9 Data Splitting

The selected features and target variable were divided into training and testing datasets using the **train-test split** technique.

An **80:20 split** was used, where **80%** of the data was allocated for training the machine learning models and the remaining **20%** was reserved for testing their performance. A **random state of 42** was specified to ensure reproducibility of the results.

The dimensions of the training and testing datasets were verified to confirm that the data was split correctly.

Outcome

The dataset was successfully partitioned into training and testing sets, providing separate data for model training and unbiased performance evaluation.

6.10 Feature Scaling

Feature scaling was performed using the **StandardScaler** to standardize the numerical features before model training.

The scaler was fitted on the training data and then applied to both the training and testing datasets. This transformation scales the features to have a mean of 0 and a standard deviation of 1, ensuring that all features are on a similar scale.

Applying the scaler only to the training data prevents data leakage and ensures a fair evaluation of the machine learning models.

Outcome

The training and testing datasets were successfully standardized and prepared for model development. This preprocessing step helps improve the performance of algorithms that are sensitive to feature scales.

6.11 Model Development

To identify the most suitable algorithm for predicting bike rental demand, multiple regression models were trained and evaluated.

The following models were implemented:

- **Linear Regression**

- Ridge Regression
- Decision Tree Regressor
- Random Forest Regressor
- XGBoost Regressor

The performance of each model was compared using regression evaluation metrics, including R^2 Score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Based on the evaluation results, the XGBoost Regressor demonstrated the best predictive performance and was selected as the final model for deployment.

6.11.1 XGBoost Regressor

The XGBoost (Extreme Gradient Boosting) Regressor was selected as the final prediction model due to its high accuracy and robust performance on the Seoul Bike Rental dataset.

Initially, the XGBoost library was imported, and an XGBRegressor model was created using the default parameters. The model was trained using the training dataset and then used to predict bike rental counts for the testing dataset.

The model's performance was evaluated using the following regression metrics:

- **R^2 Score:** Measures how well the model explains the variance in the target variable.
- **Mean Absolute Error (MAE):** Measures the average absolute difference between actual and predicted values.
- **Mean Squared Error (MSE):** Measures the average squared prediction error.
- **Root Mean Squared Error (RMSE):** Represents the prediction error in the same unit as the target variable.

Outcome

The XGBoost Regressor achieved the best performance among all the models tested and was therefore selected for hyperparameter tuning and deployment.

6.12 Hyperparameter Tuning

To further improve the performance of the selected XGBoost Regressor, hyperparameter tuning was performed using GridSearchCV.

A range of values was specified for important XGBoost parameters, including:

- **n_estimators** – Number of boosting trees.
- **max_depth** – Maximum depth of each decision tree.
- **learning_rate** – Step size used during model learning.
- **subsample** – Fraction of training samples used for each tree.
- **colsample_bytree** – Fraction of features used while constructing each tree.

A 5-fold cross-validation strategy was employed with the R^2 score as the evaluation metric to identify the optimal combination of hyperparameters. The search was executed using parallel processing (`n_jobs=-1`) to reduce computation time.

After the tuning process, the best parameter combination and the corresponding cross-validation score were obtained. The optimized XGBoost model was then used to generate predictions on the test dataset.

Outcome

Hyperparameter tuning improved the model's generalization capability by selecting the optimal parameter values. The tuned XGBoost model was chosen as the final model for performance evaluation and deployment.

6.13 Model Evaluation

The performance of the tuned machine learning models was evaluated using standard regression metrics to compare their predictive accuracy.

The following metrics were used:

- **R^2 Score** – Measures how well the model explains the variance in the target variable.
- **Mean Absolute Error (MAE)** – Measures the average absolute prediction error.
- **Mean Squared Error (MSE)** – Measures the average squared prediction error.
- **Root Mean Squared Error (RMSE)** – Represents the prediction error in the same unit as the target variable.

The performance of the following tuned models was evaluated:

- Decision Tree Regressor
- Random Forest Regressor
- XGBoost Regressor

The evaluation results were compared to identify the best-performing model. Based on the performance metrics, the **tuned XGBoost Regressor** achieved the highest predictive accuracy and was selected as the final model for deployment.

Outcome

The comparative evaluation confirmed that the tuned XGBoost model outperformed the other models, making it the most suitable choice for predicting bike rental demand.

6.13.1 Performance Comparison of Tuned Models

To compare the effectiveness of the tuned models, their performance metrics were summarized in a comparison table. The evaluation was based on **R² Score**, **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, and **Root Mean Squared Error (RMSE)**.

The comparison showed that the **tuned XGBoost Regressor** achieved the highest **R² Score (0.8749)** and the lowest error values among the three models, indicating superior predictive performance. The **Random Forest Regressor** also demonstrated strong performance, while the **Decision Tree Regressor** showed improvement after tuning but produced comparatively higher prediction errors.

Based on these results, the **tuned XGBoost Regressor** was selected as the final model for deployment.

Outcome

The comparative analysis confirmed that the tuned XGBoost model provided the best balance of accuracy and error reduction, making it the most suitable model for predicting bike rental demand.

6.14 Performance Improvement After Hyperparameter Tuning

Hyperparameter tuning significantly improved the predictive performance of the Decision Tree and Random Forest models, while the XGBoost model achieved a slight improvement over its already strong baseline performance.

A comparison of the **R² scores before and after tuning** showed that:

- **Decision Tree** improved from **0.6956** to **0.8015**.
- **Random Forest** improved from **0.7324** to **0.8676**.
- **XGBoost** improved from **0.8698** to **0.8749**, achieving the highest overall performance.

These results demonstrate the effectiveness of hyperparameter tuning in enhancing model accuracy and generalization.

6.15 Best Model Selection

Based on the evaluation metrics, the **XGBoost Regressor** was selected as the final model.

The model achieved the highest **R² Score (0.8749)** along with the lowest **MAE** and **RMSE**, making it the most accurate and reliable model for predicting bike rental demand. Its superior performance justified its selection for deployment.

6.16 Model Deployment

After selecting the tuned **XGBoost Regressor** as the final model, it was deployed as a web application using **Flask**. The trained model, along with the preprocessing components such as the **StandardScaler** and **selected features**, was saved using the joblib library to ensure consistent predictions during deployment.

A user-friendly web interface was developed where users can enter the required input values. The application processes the inputs using the same preprocessing steps applied during model training and generates real-time bike rental predictions using the trained XGBoost model.

To ensure portability and consistent execution across different environments, the Flask application was containerized using **Docker**. Docker packages the application along with all its dependencies, making deployment simpler and more reliable.

Outcome

The machine learning model was successfully deployed as a web application, allowing users to generate real-time bike rental predictions through an interactive interface. Docker containerization ensured that the application could be executed consistently across different systems.

7. Conclusion

This project successfully developed a machine learning model to predict bike rental demand using the Seoul Bike Sharing dataset. The project followed a complete machine learning pipeline, including data preprocessing, exploratory data analysis, feature engineering, feature selection, model development, hyperparameter tuning, evaluation, and deployment.

Multiple regression algorithms were implemented and compared. Among them, the **tuned XGBoost Regressor** achieved the best performance with an **R² Score of 0.8749**, making it the most suitable model for predicting bike rental demand.

The final model was deployed as a Flask web application and containerized using Docker, enabling real-time predictions through a simple and user-friendly interface. This project demonstrates the practical application of machine learning techniques to solve a real-world demand forecasting problem.

8. Future Scope

The project can be further enhanced by:

- Integrating real-time weather data through external APIs.
 - Deploying the application on cloud platforms such as AWS, Azure, or Render.
 - Developing a more interactive user interface with advanced visualization features.
 - Experimenting with advanced machine learning or deep learning models to further improve prediction accuracy.
 - Implementing automated model retraining as new bike rental data becomes available.
-